

Introduction to coroutines

1. Before you begin
2. Introduction
3. Challenges with threads
4. Coroutines in Kotlin
5. Practice on your own
6. Solution for practice exercise
7. Summary
8. Learn more

[1. Before you begin](#)

Having a responsive UI is an essential element for a great app. While you may have taken this for granted in the apps you've built so far, as you start to add more advanced features, such as networking or database capabilities, it can be increasingly difficult to write code that's both functional and performant. The example below illustrates just what can happen if long running tasks, such as downloading images from the Internet, are not handled correctly. While the image functionality works, the scrolling is jumpy, making the UI look unresponsive (and unprofessional!).



To avoid the problems with the above app, you'll need to learn a bit about something called threads. A thread is a bit of an abstract concept, but you can think of it as a single path of

execution for code in your app. Each line of code you write is an instruction that's to be executed in-order on the same thread.

You've already been working with threads in Android. Every Android app has a default "main" thread. This is (usually) the UI thread. All the code you've written so far is on the main thread. Each instruction (i.e. a line of code) waits for the previous one to finish before the next line executes.

However, in a running app, there are more threads in addition to the main thread. Behind the scenes, the processor doesn't actually work with separate threads, but rather, switches back and forth between the different series of instructions to give the appearance of multitasking. A thread is an abstraction that you can use when writing code to determine which path of execution each instruction should go. Working with threads other than the main thread, allows your app to perform complex tasks, such as downloading images, in the background while the app's user interface remains responsive. This is called concurrent code, or simply, concurrency.

In this codelab, you'll learn about threads, and how to use a Kotlin feature called coroutines to write clear, non-blocking concurrent code.

Prerequisites

- Knowledge of basic Kotlin programming concepts including loops and functions, taught in [Pathway 1: Introduction to Kotlin](#)
- How to use lambda functions in Kotlin, taught in [Pathway 3: Collections in Kotlin](#)

What you'll learn

- What concurrency is and why it's important
- How to use coroutines and Threads to write non-blocking concurrent code
- How to access the main thread to safely perform UI updates when performing tasks in the background
- How and when to use different concurrency pattern (Scope/Dispatchers/Deferred)
- How to write code that interacts with network resources

What you'll build

- In this codelab, you'll write some small programs to explore working with threads and coroutines in Kotlin

What you need

- A computer with a modern web browser, such as the latest version of [Chrome](#)
- Internet access on your computer

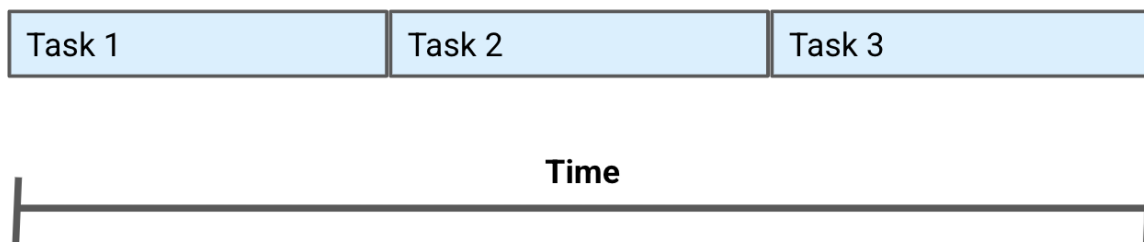
2. Introduction

Multithreading and concurrency

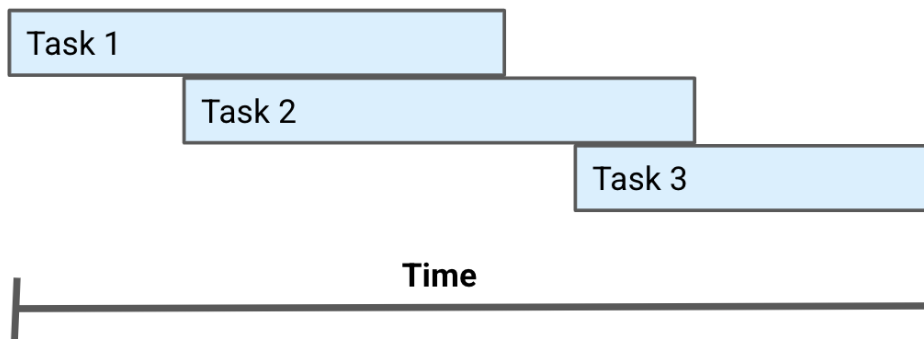
So far we have treated an Android app as a program with a single path of execution. You can do a lot with that single path of execution, but as your app grows, you need to think about concurrency.

Concurrency allows multiple units of code to execute out of order or seemingly in parallel permitting more efficient use of resources. The operating system can use characteristics of the system, programming language, and concurrency unit to manage multitasking.

Single Path of Execution



Concurrency



Why do you need to use concurrency? As your app gets more complex, it's important for your code to be non-blocking. This means that performing a long-running task, such as a network request, won't stop the execution of other things in your app. Not properly implementing concurrency can make your app appear unresponsive to users.

You'll take a look at several examples demonstrating concurrent programming in Kotlin. All the examples can be run in the Kotlin Playground:

<https://developer.android.com/training/kotlinplayground>

A thread is the smallest unit of code that can be scheduled and run in the confines of a program. Here's a small example where we can run concurrent code.

You can create a simple thread by providing a lambda. Try the following in the playground.

```
fun main() {
    val thread = Thread {
        println("${Thread.currentThread()} has run.")
    }
    thread.start()
}
```

The thread isn't executed until the function reaches the `start()` function call. The output should look something like this.

```
Thread[Thread-0,5,main] has run.
```

Note that `currentThread()` returns a `Thread` instance which is converted to its string representation which returns the thread's name, priority, and thread group. The output above might be slightly different.

Creating and running multiple threads

To demonstrate simple concurrency, let's create a couple threads to execute. The code will create 3 threads printing the information line from the previous example.

```
fun main() {
    val states = arrayOf("Starting", "Doing Task 1", "Doing Task 2", "Ending")
    repeat(3) {
        Thread {
            println("${Thread.currentThread()} has started")
            for (i in states) {
                println("${Thread.currentThread()} - $i")
                Thread.sleep(50)
            }
        }.start()
    }
}
```

Output in playground:

```
Thread[Thread-2,5,main] has started Thread[Thread-2,5,main] - Starting
Thread[Thread-0,5,main] - Doing Task 1 Thread[Thread-1,5,main] - Doing Task 1
Thread[Thread-2,5,main] - Doing Task 1 Thread[Thread-0,5,main] - Doing Task 2
Thread[Thread-1,5,main] - Doing Task 2 Thread[Thread-2,5,main] - Doing Task 2
Thread[Thread-0,5,main] - Ending Thread[Thread-2,5,main] - Ending
Thread[Thread-1,5,main] - Ending Thread[Thread-0,5,main] has started
Thread[Thread-0,5,main] - Starting
```

```
Thread[Thread-1,5,main] has started
Thread[Thread-1,5,main] - Starting
```

Output in AS(console):

```
Thread[Thread-0,5,main] has started
Thread[Thread-1,5,main] has started
Thread[Thread-2,5,main] has started
Thread[Thread-1,5,main] - Starting
Thread[Thread-0,5,main] - Starting
Thread[Thread-2,5,main] - Starting
Thread[Thread-1,5,main] - Doing Task 1
Thread[Thread-0,5,main] - Doing Task 1
Thread[Thread-2,5,main] - Doing Task 1
Thread[Thread-0,5,main] - Doing Task 2
Thread[Thread-1,5,main] - Doing Task 2
Thread[Thread-2,5,main] - Doing Task 2
Thread[Thread-0,5,main] - Ending
Thread[Thread-2,5,main] - Ending
Thread[Thread-1,5,main] - Ending
```

Run the code several times. You'll see varied output. Sometimes the threads will appear to run in sequence and other times the content will be interspersed.

Note: This invariability is caused by how the threads are executed. The scheduler gives out a slice of time to each thread and it either completes in the time period or is suspended until it receives another time slice.

3. Challenges with threads

Using threads is a simple way to start working with multiple tasks and concurrency, but are not problem-free. A number of problems can arise when you use `Thread` directly in your code.

Threads require a lot of resources

Creating, switching, and managing threads takes up system resources and time limiting the raw number of threads that can be managed at the same time. The costs of creation can really add up.

While a running app will have multiple threads, each app will have one dedicated thread, specifically responsible for your app's UI. This thread is often called the main thread or UI thread.

Note: In some cases, the UI thread and main thread may be different.

Because this thread is responsible for running your app's UI, it's important for the main thread to be performant so that the app will run smoothly. Any long-running tasks will block it until completion and cause your app to be unresponsive.

The operating system does a lot to attempt to keep things responsive for the user. Current phones attempt to update the UI 60 to 120 times per second (60 at minimum). There's a short finite time to prepare and draw the UI (at 60 frames per second, every screen update should take 16ms or less). Android will drop frames, or abort trying to complete a single update cycle to attempt to catch up. Some frames drop and fluctuation is normal but too many will make your app unresponsive.

Race conditions and unpredictable behavior

As discussed, a thread is an abstraction for how a processor appears to handle multiple tasks at once. As the processor switches between sets of instructions on different threads, the exact time a thread is executed and when a thread is paused is beyond your control. You can't always expect predictable output when working with threads directly.

For example the following code uses a simple loop to count from 1 to 50, but in this case, a new thread is created for each time the count is incremented. Think about what you'd expect the output to look like and then run the code a few times.

```
fun main() {
    var count = 0
    for (i in 1..50) {
        Thread {
            count += 1
            println("Thread: $i count: $count")
        }.start()
    }
}
```

Was the output what you expected? Was it the same every time? Here's an example output we got.

```
Thread: 50 count: 49 Thread: 43 count: 50 Thread: 1 count: 1
Thread: 2 count: 2
Thread: 3 count: 3
Thread: 4 count: 4
Thread: 5 count: 5
Thread: 6 count: 6
Thread: 7 count: 7
Thread: 8 count: 8
Thread: 9 count: 9
Thread: 10 count: 10
Thread: 11 count: 11
Thread: 12 count: 12
Thread: 13 count: 13
Thread: 14 count: 14
Thread: 15 count: 15
```

```
Thread: 16 count: 16
Thread: 17 count: 17
Thread: 18 count: 18
Thread: 19 count: 19
Thread: 20 count: 20
Thread: 21 count: 21
Thread: 23 count: 22
Thread: 22 count: 23
Thread: 24 count: 24
Thread: 25 count: 25
Thread: 26 count: 26
Thread: 27 count: 27
Thread: 30 count: 28
Thread: 28 count: 29
Thread: 29 count: 41
Thread: 40 count: 41
Thread: 39 count: 41
Thread: 41 count: 41
Thread: 38 count: 41
Thread: 37 count: 41
Thread: 35 count: 41
Thread: 33 count: 41
Thread: 36 count: 41
Thread: 34 count: 41
Thread: 31 count: 41
Thread: 32 count: 41
Thread: 44 count: 42
Thread: 46 count: 43
Thread: 45 count: 44
Thread: 47 count: 45
Thread: 48 count: 46
Thread: 42 count: 47
Thread: 49 count: 48
```

Contrary to what the code says, it looks like the last thread was executed first, and that some of the other threads were executed out of order. If you look at the "count" for some of the iterations, you'll notice that it remains unchanged after multiple threads. Even more odd, the count reaches 50 at Thread 43 even though the output suggests this is only the second thread to execute. Judging from the output alone, it's impossible to know what the final value of `count` is.

This is just one way threads can lead to unpredictable behavior. When working with multiple threads, you may also run into what's called a race condition. This is when multiple threads try to access the same value in memory at the same time. Race conditions can result in hard to reproduce, random looking bugs, which may cause your app to crash, often unpredictably.

Performance issues, race conditions, and hard to reproduce bugs are some of the reasons why we don't recommend working with threads directly. Instead, you'll learn about a feature in Kotlin called Coroutines that will help you write concurrent code.

4. Coroutines in Kotlin

Creating and using threads for background tasks directly has its place in Android, but Kotlin also offers Coroutines which provide a more flexible and easier way to manage concurrency.

Coroutines enable multitasking, but provide another level of abstraction over simply working with threads. One key feature of coroutines is the ability to store state, so that they can be halted and resumed. A coroutine may or may not execute.

The state, represented by *continuations*, allows portions of code to signal when they need to hand over control or wait for another coroutine to complete its work before resuming. This flow is called cooperative multitasking. Kotlin's implementation of coroutines adds a number of features to assist multitasking. In addition to continuations, creating a coroutine encompasses that work in a `Job`, a cancelable unit of work with a lifecycle, inside a `CoroutineScope`. A `CoroutineScope` is a context that enforces cancellation and other rules to its children and their children recursively. A `Dispatcher` manages which backing thread the coroutine will use for its execution, removing the responsibility of when and where to use a new thread from the developer.

Job	A cancelable unit of work, such as one created with the <code>launch()</code> function.
CoroutineScope	Functions used to create new coroutines such as <code>launch()</code> and <code>async()</code> extend <code>CoroutineScope</code> .
Dispatcher	Determines the thread the coroutine will use. The <code>Main</code> dispatcher will always run coroutines on the main thread, while dispatchers like <code>Default</code> , <code>IO</code> , or <code>Unconfined</code> will use other threads.

You'll learn more about these later but `Dispatchers` are one of the ways coroutines can be so performant. One avoids the performance cost of initializing new threads.

Let's adapt our earlier examples to use coroutines.

```
import kotlinx.coroutines.*

fun main() {
    repeat(3) {
        GlobalScope.launch {
            println("Hi from ${Thread.currentThread()}")
        }
    }
}
Hi from Thread[DefaultDispatcher-worker-2@coroutine#2,5,main]
Hi from Thread[DefaultDispatcher-worker-1@coroutine#1,5,main]
Hi from Thread[DefaultDispatcher-worker-1@coroutine#3`,5,main]
```

The snippet above creates three coroutines in the `Global Scope` using the default `Dispatcher`. The `GlobalScope` allows any coroutines in it to run as long as the app is running. For the reasons we

talked about concerning the main thread, this is not recommended outside example code. When you use coroutines in your apps, we will use other scopes.

The `launch()` function creates a coroutine from the enclosed code wrapped in a cancelable Job object. `launch()` is used when a return value is not needed outside the confines of the coroutine.

Let's look at the full signature of `launch()` to understand the next important concept in coroutines.

```
fun CoroutineScope.launch() {
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
}
```

Behind the scenes, the block of code you passed to `launch` is marked with the `suspend` keyword. Suspend signals that a block of code or function can be paused or resumed.

A word about runBlocking

The next examples will use `runBlocking()`, which as the name implies, starts a new coroutine and blocks the current thread until completion. It is mainly used to bridge between blocking and non-blocking code in main functions and tests. You will not be using it often in typical Android code.

```
import kotlinx.coroutines.*
import java.time.LocalDateTime
import java.time.format.DateTimeFormatter

val formatter = DateTimeFormatter.ISO_LOCAL_TIME
val time = { formatter.format(LocalDateTime.now()) }

suspend fun getValue(): Double {
    println("entering getValue() at ${time()}")
    delay(3000)
    println("leaving getValue() at ${time()}")
    return Math.random()
}

fun main() {
    runBlocking {
        val num1 = getValue()
        val num2 = getValue()
        println("result of num1 + num2 is ${num1 + num2}")
    }
}
```

`getValue()` returns a random number after a set delay time. It uses a `DateTimeFormatter`. To illustrate the appropriate entry and exit times. The main function calls `getValue()` twice and returns the sum.

```
entering getValue() at 17:44:52.311
leaving getValue() at 17:44:55.319
entering getValue() at 17:44:55.32
leaving getValue() at 17:44:58.32
result of num1 + num2 is 1.4320332550421415
```

To see this in action, replace the `main()` function (keeping all the other code) with the following.

```
fun main() {
    runBlocking {
        val num1 = async { getValue() }
        val num2 = async { getValue() }
        println("result of num1 + num2 is ${num1.await() + num2.await()}")
    }
}
```

The two calls to `getValue()` are independent and don't necessarily need the coroutine to suspend. Kotlin has an `async` function that's similar to `launch`. The `async()` function is defined as follows.

```
fun CoroutineScope.async() {
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> T
}: Deferred<T>
```

The `async()` function returns a value of type `Deferred`. A `Deferred` is a cancelable `Job` that can hold a reference to a future value. By using a `Deferred`, you can still call a function as if it immediately returns a value - a `Deferred` just serves as a placeholder, since you can't be certain when an asynchronous task will return. A `Deferred` (also called a `Promise` or `Future` in other languages) guarantees that a value will be returned to this object at a later time. An asynchronous task, on the other hand, will not block or wait for execution by default. To initiate that the current line of code needs to wait for the output of a `Deferred`, you can call `await()` on it. It will return the raw value.

```
entering getValue() at 22:52:25.025
entering getValue() at 22:52:25.03
leaving getValue() at 22:52:28.03
leaving getValue() at 22:52:28.032
result of num1 + num2 is 0.8416379026501276
```

When to mark functions as suspend

In the previous example, you may have noticed that the `getValue()` function is also defined with the `suspend` keyword. The reason is that it calls `delay()`, which is also a `suspend` function. Whenever a function calls another `suspend` function, then it should also be a `suspend` function.

If this is the case, then why wouldn't the `main()` function in our example be marked with `suspend`? It does call `getValue()`, after all.

Not necessarily. The `getValue()` function is actually called in the lambda passed into `runBlocking()`. The lambda is a suspend function, similar to the ones passed into `launch()` and `async()`. However, `runBlocking()` itself is not a suspend function. The `getValue()` function is not called in `main()` itself, nor is `runBlocking()` a suspend function, so `main()` is not marked with `suspend`. If a function does not call a suspend function, then it does not need to be a suspend function itself.

5. Practice on your own

At the beginning of this codelab, you saw the following example that used multiple threads. With your knowledge of coroutines, rewrite the code to use coroutines instead of `Thread`.

Note: You don't have to edit the `println()` statements, even though they reference `Thread`.

```
fun main() {
    val states = arrayOf("Starting", "Doing Task 1", "Doing Task 2", "Ending")
    repeat(3) {
        Thread {
            println("${Thread.currentThread()} has started")
            for (i in states) {
                println("${Thread.currentThread()} - $i")
                Thread.sleep(50)
            }
        }.start()
    }
}
```

6. Solution for practice exercise

```
import kotlinx.coroutines.*

fun main() {
    val states = arrayOf("Starting", "Doing Task 1", "Doing Task 2", "Ending")
    repeat(3) {
        GlobalScope.launch {
            println("${Thread.currentThread()} has started")
            for (i in states) {
                println("${Thread.currentThread()} - $i")
            }
        }
    }
}
```

7. Summary

You've learned

- Why concurrency is needed
- What a thread is, and why threads are important for concurrency
- How to write concurrent code in Kotlin using coroutines
- When and when not to mark a function as "suspend"
- The roles of a CoroutineScope, Job, and Dispatcher
- The difference between Deferred and Await

8. Learn more

- [Thread](#)
- [Coroutines](#)
- [Coroutines and App Performance](#)